Module 1: Foundations and Relevance

How a system decides which data sources matter for a question

Learning objectives

By the end of this module, you should be able to:

1. Describe a business question and a set of data sources in formal notation.
2. Explain the three signals that make up a hybrid relevance score: semantic, lexical and conceptual.
3. Use Bayes' rule to combine relevance with trust in each source.
4. Treat "which sources do I need?" as a set-cover problem.
5. Compute a relevance ranking by hand on a small example.

1.1 The big picture

Every business question asks the same thing: of all the data available, which pieces answer this, and how?

We write that as a single goal: find the answer A that is most probable given the question Q, the data D and our business knowledge $\mathcal{O}$.

$$A^* = \text{argmax}_A \; P(A \mid Q, D, \mathcal{O})$$

This is too hard to solve in one step, so we split it into stages. Each later module covers one stage:

Stage	Question it answers	Module
Relevance	Which sources are related to the question?	**1 (this one)**
Concept mapping	Do "sales", "rev" and "turnover" mean the same thing?	2
Implicit relationships	How do customers connect to orders?	3
Intent and entities	Is this a trend, a ranking, or a lookup? Which "Paris"?	4
Matching, confidence, ranking	Which field is which concept? How sure are we?	5
Answer selection	Which candidate answer wins?	6

1.2 The building blocks (notation)

The query

- **Q** is the raw question, for example *"What were monthly sales by region?"*

- $q = \varphi(Q) \in \mathbb{R}^d$ is its embedding: a list of d numbers that captures its meaning.

> **Intuition:** φ is a function that places text in a "meaning space". Texts with similar meanings land close together, even when they share no words.

The data sources

D = {d_1, d_2, …, d_n} is the collection of tables, reports and documents. Each source is a bundle of facts about itself:

```
dᵢ = (name, description, schema, sample values, usage metadata)
```

The ontology (business knowledge)

𝒪 = (C, R)

- **C** is the set of business concepts, such as *Revenue*, *Customer*, *Region* and *Month*.
- **R** is the set of typed relationships between them, such as *Revenue is-a Financial Metric* or *Order belongs-to Customer*.

The lexicon

L : Words → C maps everyday wording to concepts:

```
L("sales") = L("revenue") = L("rev") = Revenue
```

The concept function

C(·) returns the set of concepts that a query or source is about:

```
C("monthly sales by region") = { Revenue, Month, Region }
```

1.3 Three signals of relevance

There are three independent ways a source can "look related" to a question. Each signal catches cases the others miss.

Signal 1: Semantic similarity (meaning)

Measure the angle between the two meaning-vectors with cosine similarity:

```
cos(q, d) = (q · d) / (‖q‖ · ‖d‖)        range: −1 to 1
```

Mini example: q = (1, 1, 0) and d = (1, 0, 1).

- Dot product: 1·1 + 1·0 + 0·1 = **1**
- Norms: ‖q‖ = √2 and ‖d‖ = √2, so the product is **2**
- cos = 1 / 2 = **0.5**, which is moderately similar

Strength: catches paraphrases ("turnover" ≈ "revenue").**Weakness:** can confuse things that are related but different ("revenue" vs. "cost").

Signal 2: Lexical overlap (exact words)

BM25 is a classic keyword score. It rewards words that are shared and rare, and it adjusts for document length:

```
BM25(Q, d) = Σ_{t ∈ Q}  IDF(t) · [ f(t,d) · (k₁ + 1) ] / [ f(t,d) + k₁ · (1 − b + b · |d| / avg|d|) ]

IDF(t) = log( (N − n_t + 0.5) / (n_t + 0.5) + 1 )
```

- f(t, d) is how often term t appears in d.
- n_t is the number of sources that contain t, and N is the total number of sources.
- $k_1 \approx 1.2$ and $b \approx 0.75$ are tuning constants.

Strength: precise on exact codes and names ("SKU-4471", "EMEA").**Weakness:** blind to synonyms.

Signal 3: Concept coverage (business meaning)

What fraction of the question's concepts does the source cover?

```
coverage(Q, d) = |C(Q) ∩ C(d)| / |C(Q)|
```

Strength: grounded in curated business knowledge.**Weakness:** only as good as the ontology.

Combining them

```
rel(Q, d) = α · cos(q, d)  +  β · BM25_norm(Q, d)  +  γ · coverage(Q, d)

with  α + β + γ = 1,  all ≥ 0
```

The weights say how much to trust each signal. They can be set by hand or learned from examples of good and bad matches.

> **Why mix signals?** Each one has blind spots. A weighted mix is more robust than any single signal, much like averaging several noisy measurements.

1.4 Adding trust with Bayes' rule

Being related isn't enough. A deprecated draft table can look just as related as a certified report. We add a **prior**: a belief about how trustworthy each source is before we even look at the question.

Bayes' rule applied to sources

```
P(d | Q)  ∝  P(Q | d) · P(d)
             ──────────   ────
             "fit"        "trust"
```

Here P(Q | d) is approximated by rel(Q, d).

A model for the prior

```
P(d) ∝ exp( w₁ · curated  +  w₂ · log(1 + usage)  −  w₃ · staleness  −  w₄ · deprecated )
```

Feature	Effect	Reason
curated / certified	↑	Humans validated it
log(1 + usage)	↑ (with diminishing returns)	Popular sources are battle-tested
staleness	↓	Old data may mislead
deprecated	↓↓	Explicitly retired

> **Why the log?** Going from 0 to 100 uses is a strong signal. Going from 10,000 to 10,100 adds almost nothing. The log captures these diminishing returns.

> **Why exp?** It keeps the prior positive and turns additive evidence into multiplicative odds, the same form as logistic regression.

1.5 Worked example

Question: *"What were monthly sales by region?"*, so **C(Q) = {Revenue, Month, Region}**

Candidate sources:

Source	Description	Concepts covered
d_1 `orders_fact`	Line-level orders with dates and amounts	Revenue, Month
d_2 `regional_revenue_summary` (certified)	Monthly revenue rolled up by region	Revenue, Month, Region
d_3 `employee_directory`	Staff names, roles and office region	Region

Weights: α = 0.5, β = 0.2, γ = 0.3

Step 1: Raw signals

Source	cos	BM25_norm	coverage
d_1	0.72	0.40	2/3 ≈ 0.667
d_2	0.85	0.90	3/3 = 1.000
d_3	0.20	0.05	1/3 ≈ 0.333

Step 2: Combined relevance

```
rel(d₁) = 0.5(0.72) + 0.2(0.40) + 0.3(0.667) = 0.360 + 0.080 + 0.200 = 0.640
rel(d₂) = 0.5(0.85) + 0.2(0.90) + 0.3(1.000) = 0.425 + 0.180 + 0.300 = 0.905
rel(d₃) = 0.5(0.20) + 0.2(0.05) + 0.3(0.333) = 0.100 + 0.010 + 0.100 = 0.210
```

Step 3: Apply priors and normalize

Assume the priors are $P(d_1) = 0.3$, $P(d_2) = 0.5$ (certified) and $P(d_3) = 0.2$.

Source	rel × prior	Posterior P(d | Q)
d_1	0.640 × 0.3 = 0.192	0.192 / 0.6865 ≈ **0.28**

d_2	0.905 × 0.5 = 0.4525	0.4525 / 0.6865 ≈ **0.66**
d_3	0.210 × 0.2 = 0.042	0.042 / 0.6865 ≈ **0.06**

The normalizer is 0.192 + 0.4525 + 0.042 = 0.6865.

Result: d_2 wins clearly. d_1 stays a credible backup, because it could be aggregated to answer the question. d_3 shares only the word "region" and is effectively ruled out.

1.6 When one source isn't enough: set cover

Some questions span several concepts that no single source holds. Change the question to *"monthly sales by region and product category"*:

```
C(Q) = { Revenue, Month, Region, ProductCategory }
```

If no source covers all four, we need the **smallest set of sources whose combined coverage includes every needed concept**:

```
S* = argmin_{S ⊆ D} |S|    subject to    C(Q) ⊆ ⋃_{d ∈ S} C(d)
```

This is the classic **set-cover problem**. Finding the exact solution is NP-hard, but a greedy approach works well in practice:

```
Greedy set cover:
  U ← C(Q)                        (concepts still needed)
  S ← ∅
  while U ≠ ∅:
      d* ← argmax_d  |C(d) ∩ U| · P(d | Q)     (most new coverage, weighted by trust)
      S ← S ∪ {d*}
      U ← U \ C(d*)
  return S
```

Applied:

1. Pick d_2, which covers {Revenue, Month, Region}. Still needed: {ProductCategory}.
2. Pick d_4 `product_dim`, which covers {ProductCategory}. Still needed: ∅.
3. **$S^* = \{d_2, d_4\}$.** How to connect them is the subject of Module 3.

> **Guarantee:** greedy set cover is within a factor of $\ln|C(Q)| + 1$ of the optimal solution. For a 4-concept question, that means at most about 2.4× as many sources as the best possible answer, and usually far fewer.

1.7 Common pitfalls

Pitfall	Symptom	Fix in the math
Relying only on semantic similarity	Picks "cost" tables for "revenue" questions	Add lexical and coverage terms ($\beta, \gamma > 0$)

Relying only on keywords	Misses "turnover" when the table says "revenue"	Add semantic similarity ($\alpha > 0$)
Ignoring trust	Picks a deprecated draft table	Multiply by the prior P(d)
Single-source assumption	Can't answer questions that span several concepts	Solve set cover
Unbounded popularity	One huge table dominates everything	Use log(1 + usage), not raw usage

Check your understanding

1. **Compute:** given q = (2, 0, 1) and d = (1, 1, 1), what is cos(q, d)?
2. **Reason:** two sources have identical relevance, but one is certified and the other is deprecated. Which part of the formula separates them?
3. **Apply:** if $\gamma = 0$, what kind of mistake becomes more likely?
4. **Extend:** in the worked example, raise d_1's prior to 0.6 and lower d_2's to 0.2, keeping d_3 at 0.2. Does the ranking change?

<details>
<summary>Answers</summary>

1. $q \cdot d = 2 + 0 + 1 = 3$; $\|q\| = \sqrt{5} \approx 2.236$; $\|d\| = \sqrt{3} \approx 1.732$; cos ≈ 3 / 3.873 ≈ **0.775**
2. The **prior P(d)**: the curated (+) and deprecated (−) terms in the exponent.
3. Sources that sound right but don't hold the needed business concepts get selected. In other words, there's no check against the ontology.
4. d_1: 0.640 × 0.6 = 0.384. d_2: 0.905 × 0.2 = 0.181. **Yes, d_1 now ranks first.** Priors can override relevance, which is why they need careful calibration.

</details>

Key takeaways

- A question and its sources become **vectors, word sets and concept sets**, and relevance compares all three.
- **Hybrid scoring** balances meaning, exact wording and business concepts.
- **Bayes' rule** turns "looks related" into "is related *and* trustworthy".
- **Set cover** decides how many sources a question needs.

Coming next: Module 2, Concept Mapping

How do "sales", "rev", "turnover" and code "R01" all collapse into one concept? We'll cover equivalence classes, probabilistic disambiguation with softmax, noisy-channel abbreviation expansion, PMI for codes, and semantic distance in an ontology graph.